Nedim Omanović

Week 2 Prompt Iteration Log

1 Task and FL-01 Context

The real task selected from my FL-01 workflow audit is planning an interactive 2D avatar for my developer portfolio. The avatar should be a semi-realistic illustration based on my own photograph, fit a dark interface with green accents, follow the cursor with its eyes on desktop, and smile with a thumbs-up gesture when the Contact section becomes visible. It must also work on mobile and respect reduced-motion preferences.

The goal of this exercise is to investigate how prompt engineering changes the usefulness, specificity, and structure of an AI response. The supplied conversation exports contain responses from ChatGPT and Claude to prompts about this project. The outputs below are concise evidence-based summaries of those responses, not verbatim reproductions of every paragraph in the original conversations.

2 Prompt Iteration Log

2.1 V0 – Naive prompt

Technique: Baseline prompt without role, context, examples, or output constraints.

```
Help me build an interactive avatar for my developer portfolio.
```

Observed output: The response proposed a single-file avatar demo with cursor/head movement, blinking, mood changes, a speech bubble, and optional glasses/headphones and hoodie colours. It assumed a friendly, flat illustration and placeholder identity details. It also acknowledged that mobile behavior had not been tested.

What changed and why: This baseline was broad and allowed the model to choose the visual style and interaction details. The result was useful for brainstorming, but it did not fully match the later requirement for a semi-realistic, photo-based illustration and a Contact-triggered thumbs-up.

2.2 V1 – Role assignment

Technique: Assigning a relevant expert role.

```
Act as a senior frontend developer mentoring a junior developer.
Help me design an interactive 2D avatar for my React developer portfolio.
Recommend a practical implementation approach and explain the main technical decisions.
```

Observed output: The response shifted toward implementation planning and mentoring. It discussed component separation, React integration, cursor interaction, and possible animation approaches rather than treating the task only as a visual concept.

What changed and why: The role encouraged a more technical and explanatory response. However, the prompt still lacked the full visual requirements, mobile constraints, accessibility requirements, and a fixed response structure.

2.3 V2 – Context and motivation

Technique: Providing project context, constraints, and purpose.

```
Act as an experienced frontend developer specializing in React and UI animation.
```

```
I am building a personal developer portfolio to present my skills and projects to
    recruiters and potential employers. I want a semi-realistic 2D avatar based on my
    own photograph, with a dark visual theme and green accents.

The avatar should follow the visitor's cursor with its eyes on desktop. When the
    visitor reaches the Contact section, it should smile and give a thumbs-up gesture.
    The experience must also work well on mobile devices and respect reduced-motion
    preferences.

My goal is to create a polished, lightweight feature that I can realistically implement
     and maintain as a junior developer.

Recommend a practical technical approach. Explain the components, animations,
    responsive behavior, accessibility considerations, and implementation challenges. Do
     not assume the avatar has already been implemented.
```

Observed output: The answer became more aligned with the intended portfolio. It recommended separating the artwork from interaction logic, preparing independently animated layers, using pointer capability detection, observing the Contact section, and supporting reduced motion.

What changed and why: Context reduced ambiguity. The answer addressed the actual visual style and behavior instead of inventing unrelated avatar controls. It also surfaced dependencies that the naive prompt did not mention, especially the need for separately animated artwork layers.

2.4 V3 – Few-shot examples

Technique: Showing examples of the desired level of specificity.

```
Act as an experienced React frontend developer. Help me plan an interactive 2D avatar
    for my developer portfolio.

Example 1:
Input: "The avatar should react to the cursor."
Good response: "Track the cursor position relative to the avatar container, calculate a
     bounded horizontal and vertical offset, and apply that offset to the pupils. Clamp
    the movement so the eyes remain visually natural."

Example 2:
Input: "The avatar should react when the visitor reaches Contact."
Good response: "Observe the Contact section with IntersectionObserver and trigger a
    friendly expression when the section becomes visible. Prevent the animation from
    repeatedly restarting during minor scrolling."

Apply the same level of practical detail to this project:
- Semi-realistic 2D illustration based on my photograph.
- Dark portfolio theme with green accents.
- Eyes follow the cursor on desktop.
- Smile and thumbs-up when Contact becomes visible.
- Responsive desktop and mobile behavior.
- React, TypeScript, SVG, and lightweight animation where appropriate.
- Reduced-motion support.

Suggest the architecture and explain how each requirement could be implemented.
    Distinguish recommendations from work that has actually been implemented.
```

Observed output: The response provided concrete implementation concepts: bounded pupil movement, `IntersectionObserver`, separation of artwork and interaction logic, responsive behavior, and cleanup of listeners and observers. It also explicitly distinguished proposed code from

implemented code.

What changed and why: The examples demonstrated what "practical detail" meant. The output contained more actionable techniques instead of vague statements such as "make the avatar interactive." Few-shot examples guided response style but did not guarantee that every code fragment would be complete or tested.

2.5 V4 – Output structure

Technique: Specifying the required response format.

```
Act as a senior frontend developer mentoring a junior developer.

Design an interactive 2D avatar for my React developer portfolio. It should be a semi-
    realistic illustration based on my own photograph, use a dark theme with green
    accents, follow the cursor with its eyes on desktop, and smile with a thumbs-up when
     the Contact section becomes visible. It must work on mobile and respect reduced-
    motion preferences.

Organize the response using exactly these sections:
1. Recommended Approach
2. Technology Stack
3. Component Architecture
4. Animation Logic
5. Desktop and Mobile Behavior
6. Accessibility and Performance
7. Implementation Steps
8. Risks and Limitations
9. First Coding Step

Keep the response practical and specific. Do not claim anything has already been
    implemented, and clearly state assumptions.
```

Observed output: The response followed the nine requested sections. It proposed a layered illustration, React and TypeScript, pointer tracking, `IntersectionObserver`, mobile-specific placement, reduced-motion support, performance precautions, and a small first coding step.

What changed and why: The fixed headings made the answer easier to review and compare. They also reduced the chance of omitting accessibility, risks, or the first implementation step. A structured answer was more usable as a development plan, although headings alone do not verify the correctness of the proposed code.

2.6 V5 – Step decomposition

Technique: Breaking the larger task into ordered phases.

```
Act as an experienced React and TypeScript developer who mentors junior developers.

Help me plan and begin implementing an interactive 2D avatar for my developer portfolio
    .

Context:
- Semi-realistic illustration based on my own photograph.
- Dark theme with green accents.
- Eyes follow the cursor on desktop.
- Smile and thumbs-up when the Contact section becomes visible.
- Responsive desktop and mobile behavior.
- Respect reduced-motion preferences.
- Prefer React, TypeScript, SVG, and lightweight animation.
- Keep the implementation maintainable and incremental.
```

```
Work through these phases in order:
1. Requirements: summarize requirements and state reasonable assumptions.
2. Architecture: propose components, file organization, states, and responsibilities.
3. Visual asset: explain how to prepare layered artwork without claiming it already
    exists.
4. Interaction design: explain cursor tracking, Contact detection, touchscreen behavior
    , and avoiding repeated triggers.
5. Implementation: give an ordered plan and code for the first meaningful step, with
    filenames and dependencies.
6. Testing: provide a checklist for desktop, mobile, accessibility, reduced motion, and
     edge cases.

Use clear headings for all six phases. Keep code realistic and maintainable. Identify
    assumptions and unresolved decisions. Do not invent files, installed packages,
    completed work, or test results.
```

Observed output: The response separated requirements, architecture, visual assets, interaction behavior, implementation, and testing. It recommended building the artwork and static component first, then adding cursor tracking and Contact detection separately. It included edge cases such as observer cleanup, missing Contact elements, mobile behavior, and reduced-motion preferences.

What changed and why: Decomposition converted a broad design request into smaller milestones that can be implemented and verified. It also made dependencies clearer: convincing eye movement and a thumbs-up require suitable layered artwork, not only React code. The response remained a plan; its code and recommendations still require integration and testing in the actual repository.

3 Final Prompt: Cross-Model Test

The following final prompt was run against ChatGPT and Claude. The response summaries below are based on the supplied conversation exports.

```
Act as an experienced React and TypeScript developer who mentors junior developers.

Help me design and begin implementing an interactive 2D avatar for my developer
    portfolio. The avatar should be a semi-realistic illustration based on my own
    photograph, use a dark theme with green accents, follow the cursor with its eyes on
    desktop, and smile with a thumbs-up gesture when the Contact section becomes visible
    . It must work on mobile and respect reduced-motion preferences.

Please provide:
1. Recommended architecture and component responsibilities.
2. A suitable technology stack and reasons for each choice.
3. An approach to layered artwork and facial features.
4. Cursor-tracking logic with bounded pupil movement.
5. Contact-section detection and state transitions.
6. Desktop/mobile behavior and reduced-motion support.
7. A phased implementation plan and a small first coding step.
8. Testing checks, risks, limitations, and realistic alternatives.

Use clear headings. Distinguish proposed code from implemented work. Do not invent
    repository details or claim that code has been tested.
```

3.1 ChatGPT response – observed result

ChatGPT recommended a reusable React/TypeScript component with the artwork separated from interaction logic. Its suggestions included SVG or aligned transparent layers, bounded eye movement, `IntersectionObserver` for Contact visibility, pointer-capability detection, reduced-motion support, and a staged implementation plan. It included example TypeScript/React snippets and a testing checklist. The response emphasized cleaning up observers and event listeners, avoiding unnecessary re-renders during pointer movement, and verifying the integration in the real project.

Strengths: Practical implementation details, clear component responsibilities, and attention to accessibility and lifecycle cleanup.

Limitations: The proposed snippets are not a complete application and were not verified against the user's actual repository. The visual asset still has to be created and correctly layered before independent eye and hand animation can work.

3.2 Claude response – observed result

Claude recommended a layered illustration rendered as a reusable component, with a base portrait, separately animated eyes, alternate mouth states, and a thumbs-up layer. It suggested CSS custom properties and a small `requestAnimationFrame` loop for cursor tracking, `IntersectionObserver` for Contact visibility, and capability queries to distinguish fine-pointer devices from touch devices. It also proposed a component and hook structure, a phased implementation checklist, and accessibility and performance precautions.

Strengths: Detailed low-level animation guidance, including clamped pupil offsets, avoiding React renders on every pointer event, reducing motion, and keeping the avatar from blocking controls.

Limitations: Some implementation choices assume a particular setup (for example, React/Vite and layered raster assets) and must be adapted to the actual Next.js portfolio. The response itself noted that the code and asset structure were proposed and not tested on a phone.

4 Cross-Model Comparison

Dimension	ChatGPT	Claude
Tone	Mentoring and implementation-oriented; explains component responsibilities and integration steps.	Technical and direct; gives detailed low-level animation guidance and explicit assumptions.
Structure	Organized sections, phased steps, and a test checklist.	Organized architecture, technology choices, animation logic, risks, and implementation sequence.
Technical focus	React component boundaries, Contact visibility, reduced motion, and integration/testing.	CSS variables, `requestAnimationFrame`, pointer capability detection, and layered raster assets.
Accuracy and caution	Clearly treats snippets as examples and calls for repository-level verification.	Also states that the proposal is untested and identifies visual-layer alignment and mobile testing as risks.
Failure points	A suggested hook or snippet may need adaptation to the actual component lifecycle and repository structure.	The proposed asset pipeline and Vite assumptions may not match a Next.js project without adjustment.

Neither model's response should be treated as verified production code. Both correctly identify that the avatar's artwork is a major dependency: a single flat image cannot convincingly support independent pupil movement and a separate thumbs-up pose without additional layers or alternate poses. The most useful next step is to inspect the real portfolio repository, prepare the artwork, and implement one feature at a time while testing it.

5 What the Iterations Demonstrate

The prompt improvements added different kinds of control:

- **Role assignment** encouraged an expert, mentoring perspective.
- **Context and motivation** tied the response to the actual portfolio, target audience, style, and constraints.
- **Few-shot examples** demonstrated the desired level of practical specificity.
- **Output structure** made the answer easier to review and reduced the chance of missing key areas.
- **Step decomposition** turned a broad task into smaller implementation and testing milestones.

The main lesson is that a better prompt does not automatically make code correct. It makes the model's assumptions, scope, and expected output clearer, while verification remains the developer's responsibility.

6 Reusable Prompt Template

This template can be reused by someone who has a different project. Replace the bracketed fields with project-specific information.

```
Act as a [relevant expert role] mentoring a [skill level] developer.

I am working on [project/task] for [audience or purpose].
The desired outcome is [specific outcome].
Requirements:
- [functional requirement 1]
- [functional requirement 2]
- [platform, technology, or environment constraints]
- [accessibility, performance, or maintenance constraints]

Please work through these phases:
1. Summarize the requirements and state assumptions.
2. Recommend an architecture and explain the key decisions.
3. Break the work into small, ordered implementation steps.
4. Provide the first meaningful implementation step, including filenames
   and code where appropriate.
5. Explain how to test the result and identify likely failure points.

Use these output sections: [required headings].
If useful, follow this example of the desired level of detail: [short example].
Distinguish verified facts from assumptions and proposed code. Do not invent
existing files, dependencies, test results, or completed work.
```

7 Conclusion

The iteration process improved the prompt from a vague request into a constrained, testable engineering task. Context and examples improved relevance; output structure improved readability; and decomposition improved the implementation sequence. The cross-model comparison also showed that both systems can provide useful technical plans, but neither removes the need to verify assumptions, test code, and prepare suitable visual assets.

A Screenshots as Evidence of Model Runs

The screenshots below document the conversations used for the cross-model comparison. Place the image files in the same directory as this `.tex` file. Each image is inserted directly rather than as a floating figure, so its heading will not be left behind on a previous page. The images are scaled proportionally to fit the page.

A.1 Claude conversation

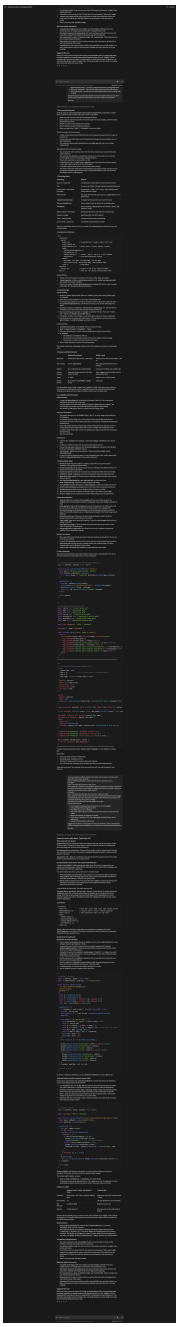


Figure 1: Claude conversation screenshot.

A.2 ChatGPT conversation – screenshots 1a and 1b



Figure 2: ChatGPT conversation screenshot 1a (upper part).



Figure 3: ChatGPT conversation screenshot 1b (lower part).

A.3 ChatGPT conversation – screenshot 2

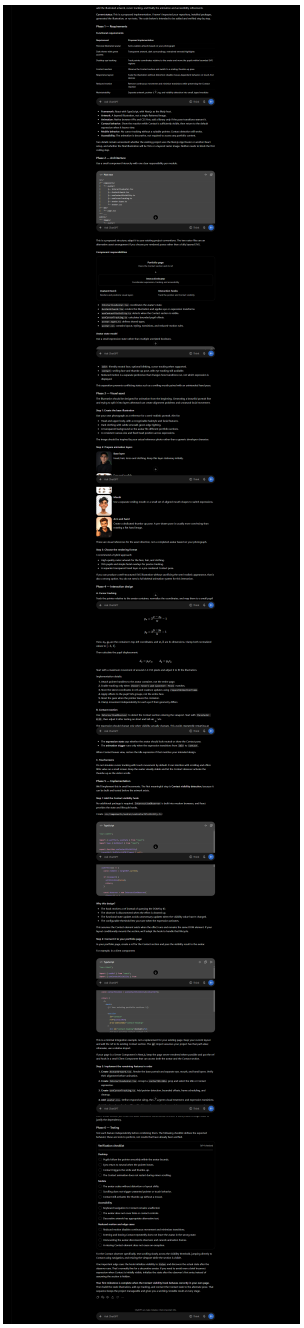


Figure 4: ChatGPT conversation screenshot 2.

Evidence note: These figures document the conversations only to the extent that the screenshots show the prompts and model responses. They do not independently prove that every intermediate iteration V0–V5 was run unless those iterations are visible in the screenshots or preserved elsewhere.